

Lecture Notes: Data Access Layer and Service Layer

1. Introduction to Data Access Layer

Data Access Layer:

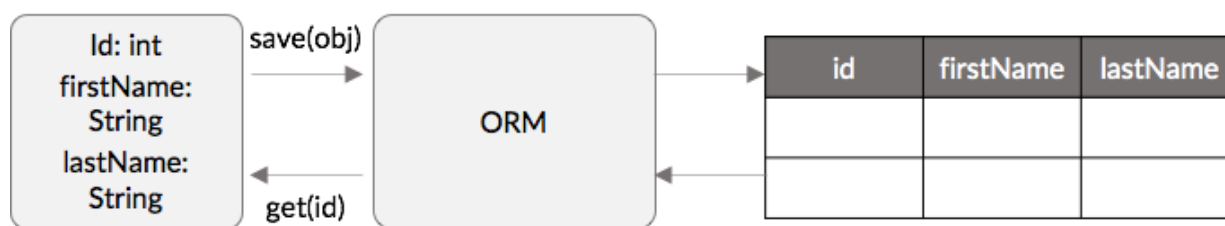
This layer is a part of the back-end segment. All the components (basically, beans) that are responsible for interacting with the database are part of this layer.

The components inside this layer are not dependent on the components of the other two layers.

ORM:

Object Relational Mapping

According to Wikipedia, “Object-relational mapping (ORM, O/RM, and O/R mapping tool) in computer science is a programming technique for converting data between incompatible type systems using object-oriented programming languages”.



Java Object

External Library (Jar)

RDBMS Table

- Takes no extra effort; simply add as dependency
- Extremely less chances of bugs
- Well-tested
- Just mention which database you are using; it will adapt accordingly

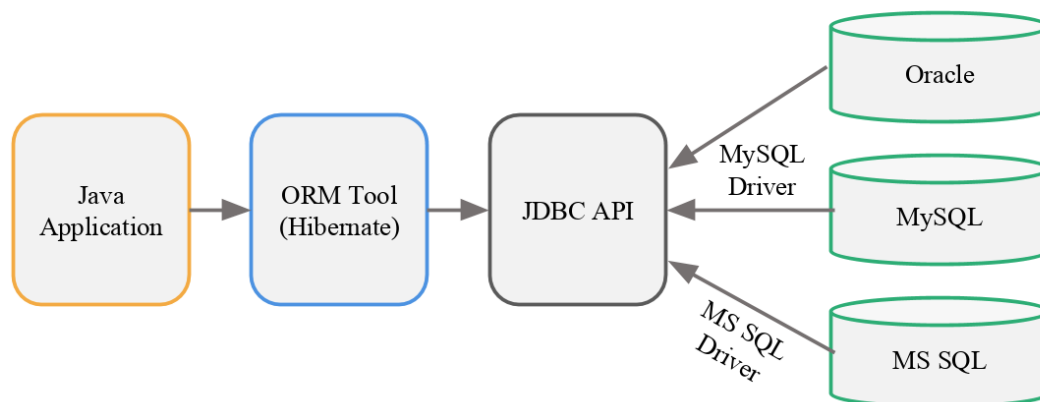
ORM allows easy transfer of data between RDBMS tables and the objects that are part of the application code. ORM helps in converting data between Java objects and RDBMS table rows and provides other features, such as table creation based on the entities present in the application.

A lot of tasks that require developers' efforts while working with the plain JDBC API can be done automatically by ORM.

However, ORM does not replace JDBC API but uses it internally. For example, when fetching data from the RDBMS table, ORM uses the SELECT query of JDBC API, gets the ResultSet and maps it to Collection objects.

It supports primary key generation strategies based on the underlying RDBMS. It also supports database neutral SQL queries to operate on RDBMS.

Examples of Java ORM frameworks are Hibernate (for RDBMS) and Toplink (for both RDBMS and non-RDBMS). In this course, we will be using Hibernate.



Thus, a Java application would be dependent on an ORM tool; an ORM tool would be dependent on JDBC API, and different drivers would also be dependent on JDBC API.

The Dependency Inversion Principle states that high-level modules (ORM) should not be dependent on low-level modules (JDBC Drivers), but both should be dependent on the abstraction (JDBC API). Hence, JDBC API provides only the interfaces, which are implemented by the JDBC Drivers.

Thus, ORM uses JDBC API to interact with the RDBMS, and we can plug different JDBC Drivers based on the RDBMS that we are using.

In future, if we want to change the RDBMS, we only need to unplug the current driver and plug the new driver, and ORM will manage the rest.

JPA

Java Persistence API

Java Specification: Specifications in Java define a set of interfaces, method declarations or annotations provided by the Java technology as a standard set of rules.

Vendor companies must provide code implementation according to specifications,

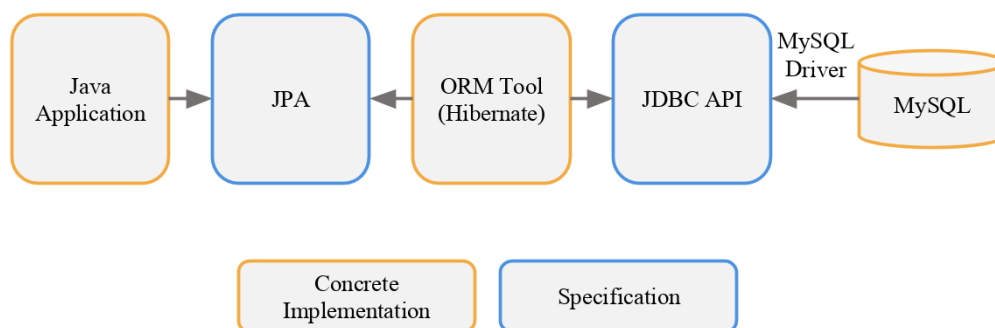
which ensures uniformity between different implementations.

Some examples of specifications are as follows:

- JDBC Specification: It is implemented by different RDBMS vendors, such as Oracle and MySQL.
- Servlet Specification: It is implemented by different Servlet containers such as Tomcat and JServ.

Java provides a specification for ORM, and that specification is called JPA (Java Persistence API).

One should not write code to implementations (concrete classes) but to specifications (interfaces). This means that the code should not contain any reference to the concrete classes. The properties, method parameters and return types of methods should be of the 'interface' type.



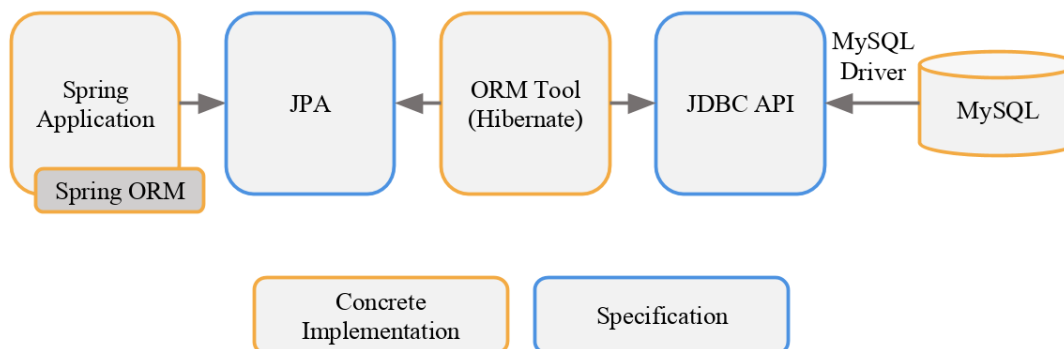
JPA is a specification for an ORM technique, and Hibernate is one of its implementations, which is an ORM tool.

Similarly, JDBC API is a specification to connect Java applications with RDBMS, and MySQL Connector/J is one of its implementations, which is a JDBC Driver that helps your Java application connect with MySQL Database.

Spring ORM

Spring ORM is a Spring module that helps you integrate your Spring applications with JPA or Hibernate such that you can use the ORM tools using IoC and DI.

It is not easy to integrate Spring applications with JPA or Hibernate and use them with all the benefits provided by Spring, such as IoC and DI. This is where Spring ORM comes into play.



Spring-ORM provides simplified templates for accessing databases through ORM technologies, such as Hibernate, My(i)Batis etc.

It is an umbrella module that covers many persistence technologies, namely JPA, JDO, Hibernate and iBatis. For each of these technologies, Spring provides integration classes so that each technology can be used following Spring principles of configuration and smoothly integrates with Spring transaction management.

Database Set up:

To set up the database, we need the following dependencies:

- JPA specification
- Hibernate
- Spring ORM
- JDBC API
- MySQL JDBC Driver

The first three dependencies can be added using only one starter POM, `spring-boot-starter-data-jpa`.

JDBC API is part of JDK, so there is no need to add it externally.

MySQL JDBC Driver can be added using `mysql-connector-java` dependency.

```

<dependency>

  <groupId>org.springframework.boot</groupId>

  <artifactId>spring-boot-starter-data-jpa</artifactId>

</dependency>

<dependency>

  <groupId>mysql</groupId>
  
```

```
<artifactId>mysql-connector-java</artifactId>  
  
<scope>runtime</scope>  
  
</dependency>
```

Configure the database through application.properties:

```
spring.datasource.url=jdbc:mysql://localhost:3306/TEST  
spring.datasource.username=root  
spring.datasource.password=root  
  
spring.jpa.show-sql=true  
spring.jpa.hibernate.ddl-auto=create  
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQL5Dialect
```

The **spring.jpa.hibernate.ddl-auto** property is used to select the DDL mode for Hibernate. It can have one of the following values:

- create: It drops the existing tables and then creates the new ones.
- update: It updates the existing tables with the new changes done in the entity classes.
- create-drop: It creates and drops the tables in the end.
- validate: It matches the tables with the entity classes. If they do not match, then it throws an error.
- none: It turns off the DDL mode. This is also the default value.

The **spring.jpa.properties.hibernate.dialect** property is used by Hibernate to decide for which database it has to generate queries. As we are using MySQL Database, we need to use MySQL5Dialect.

Entities

In RDBMS, entities are objects that are stored as rows in the tables.

Similarly, in JPA, entities are POJO (Plain Old Java Object) classes whose objects you want to save in the rows of the tables.

For example, we want to save customer data into the database. Then, there would be a POJO class named 'Customer' in our Movie Booking Application.

Entities in an application can be created using the following two different

approaches:

- Database-first
- Entity-first

In the Database-first approach, we first create the schema in the database. Then, we create the entire classes in the application. So, the table name would be the class name, and the columns would be the attributes of the class.

In the Entity-first approach, we first create the entities in the application. Then, we use ORM tools, such as Hibernate, to create the schema for the application. So, the class name would be the table name, and the attributes of the class would be the columns in the table.

To save the objects of the POJO class into the database, queries are not to be written. Hibernate maps this class to the table present in the database. To do that, metadata about the class has to be provided, which is done by using the **@Entity** annotation. It is used to mark POJO classes whose objects are to be saved in the rows of the database tables.

On application start, Hibernate scans all the classes that are marked with the **@Entity** annotation and maps them to the tables present in the database. When choosing **create** as the DDL mode, Hibernate drops all the tables corresponding to the entities and creates new ones on application start.

Every JPA entity must have a primary key, in the same way as every table in the database has a primary key, and to make any attribute of the entity class its primary key, mark that attribute with the **@Id** annotation.

Thus, to create an entity class that can be mapped to a database table by Hibernate, there are two requirements as given below.

- That class should be marked with the **@Entity** annotation.
- One of its fields should be marked with the **@Id** annotation.

To make the table name to be different from the name of the entity class, mark the entity class with the **@Table** annotation.

The table name can be provided using the **name** attribute of the **@Table** annotation.

To apply constraints to the columns, use the **@Column** annotation.

Some of the most useful attributes of the **@Column** annotation are as follows:

- **name**: To provide a custom name to the column inside the database table
- **length**: To specify the length of the column datatype
- **nullable**: To specify whether or not the column value can be null
- **unique**: To specify whether or not the column value is unique

To specify how the primary key would be generated, use the **@GeneratedValue** annotation. Specify the primary key generation strategy using the **strategy** attribute of the **@GeneratedValue** annotation.

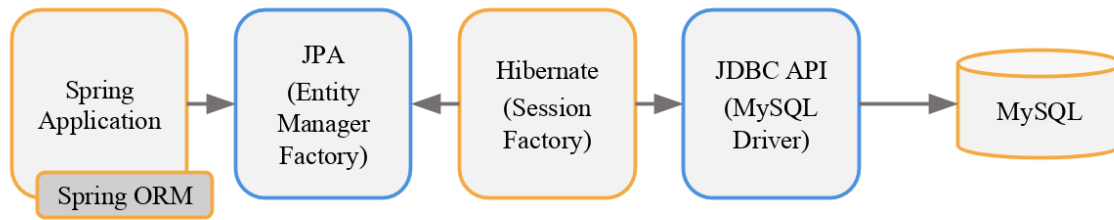
The value of the strategy attribute can be one of the following four types:

- IDENTITY: It uses a database identity column (auto-increment). It is mainly used with MS SQL or MySQL.
- SEQUENCE: It uses a database sequence. It is mainly used with Oracle or PostgreSQL.
- TABLE: It uses an underlying database table to ensure uniqueness. It works with all databases.
- AUTO: It uses an appropriate strategy for the underlying database. This is the default in case no strategy is specified. It is also the preferred type, as the ORM tool will choose the best strategy for the underlying database.

```
@Entity
public class Movie {
    @Id
    @GeneratedValue(strategy = GenerationType.AUTO)
    private int movieId;
    @Column(length = 50, nullable = false, unique = true)
    private String movieName;
    @Column(name = "movie_desc", length = 500, nullable = false)
    private String movieDescription;
    @Column(nullable = false)
    private LocalDateTime releaseDate;
    @Column(nullable = false)
    private int duration;
    @Column(length = 500, nullable = false)
    private String coverPhotoUrl;
    @Column(length = 500, nullable = false)
    private String trailerUrl;
}
```

2. Creating Repositories

When you run the Spring Boot application with the required dependencies (spring-boot-starter-data-jpa and mysql-connector-java) and database configuration (provided in the application.properties file), Spring Boot does the autoconfiguration and provides you with the following set-up.



Spring Boot creates an **EntityManagerFactory** bean, which is used to get the **EntityManager** to exchange the entities with the RDBMS database.

EntityManagerFactory and EntityManager are parts of the JPA specification, so it's just a wrapper around the Hibernate objects that do the actual work in the background.

SessionFactory and **Session** are provided by the Hibernate, which are used by EntityManagerFactory and EntityManager to save and retrieve the entities to and from the database.

Essentially, a Session is used to get a physical connection with a database. The Session object is lightweight and designed to be instantiated each time an interaction is needed with the database. Persistent objects are saved and retrieved through a Session object.

SessionFactory is a factory class for Session objects. It is available for the whole application, while a Session is only available for a particular transaction.

The main function of the Session is to offer, create, read, and delete operations for instances of mapped entity classes.

EntityManager	Session
It is a part of JPA Specification that makes your code loosely coupled with ORM tools.	It is a part of Hibernate that makes your code tightly coupled with Hibernate.
Internally, it uses Session to store and retrieve entities.	Internally, it uses the JDBC API to store and retrieve entities.
It provides less control over CRUD operations.	It provides more control over CRUD operations.
It provides CRUD methods such as persist(), find(), merge(), remove(), etc.	It provides CRUD methods such as persist(), save(), saveOrUpdate(), get(), load(), merge(), update(), delete(), etc.

The **@Repository** annotation is a special form of the **@Component** annotation.

Similar to the **@Component** annotation, any class marked with the **@Repository** annotation will be instantiated as a Spring bean by Spring. A special feature of this annotation is that it converts checked JDBC exceptions into unchecked Spring exceptions.

Entities (or entity objects) can be in one of the following three states:

- **Transient:** Newly created entity objects that do not have corresponding rows in the database table
- **Persistent:** Entity objects that have corresponding rows in the database table once the transaction is committed
- **Detached:** Entity objects that have corresponding rows in the database table but are not in the persistent state because the session is probably closed

Session interface provides multiple methods, some of which are :

- **save()** - Persist the given transient instance, first assigning a generated identifier.
- **persist()** - Persist the given transient instance. However, It does not return anything. Its void return type.
- **load()** - Fetch persistent instance from the database for the given identifier or exception if an instance is not found.
- **get()** - Return the persistent instance of the given named entity with the given identifier or null if there is no such persistent instance.
- **delete()** - Remove a persistent instance from the datastore.

Note that using the **session.delete()** method, we can only delete entities that are in the persistent state. If we want to delete entities that are in the detached state, then we will need to first move them back to the persistent state using the **merge()** method.

We cannot delete entities that are in the transient state because they do not have any corresponding rows that can be deleted from the database.

The difference between **save()** and **persist()** methods of Session are:

save()	persist()
It returns the generated ID.	It returns the void.
It also works outside transaction boundaries.	It only works inside transaction boundaries.
It saves detached entities as new rows.	It throws exceptions when we try to save detached entities.

It can only be used with session objects.	It is internally called by the JPA EntityManager.
---	---

Hibernate provides two methods to retrieve entities from the database, `get()` and `load()`.

get()	load()
If an entity is not found for the given ID, then the method returns null.	If an entity is not found for the given ID, then the method throws an exception.
It is slower than the <code>load()</code> method, as it fully initialises objects, including foreign key mapped objects.	It is faster than the <code>get()</code> method, as it does not fully initialise objects.

There are certain issues that may face while working with EntityManager and Session, which are as follows:

- You need to write boilerplate code, such as opening and closing sessions and starting and committing transactions.
- These classes only provide methods to perform basic CRUD operations; you will need to write JPQL or HQL queries for other operations.
- You need to manage entity states manually; otherwise, exceptions are thrown or can lead to duplication of data.

Spring Data

Spring Data helps you deal with these issues. It is a Spring Project, which provides a common interface to access data from both RDBMS databases and NoSQL databases. It not only provides a common interface but also makes data access so easy that you can create the Data Access Layer using only DAO interfaces.

With Spring Data, you simply need to create DAO interfaces. You need not provide any implementations for these interfaces; Spring Data does that.

Spring Data can provide several sub-projects based on the requirement. In this course, we have used **Spring Data JPA** to create the Data Access Layer.

While using Spring Data JPA, we can create the entire Data Access Layer using only DAO interfaces. The only requirement is that the DAO interface should extend the **JpaRepository** interface. With this, you will have methods for all the basic CRUD operations.

```
public interface MovieDao extends JpaRepository<Movie, Integer> {}
```

The JpaRepository interface is of the generic type, which accepts two types of parameters.

- The first parameter specifies the type of Entity for which we are providing the DAO interface.
- The second parameter specifies the type of the primary key for that entity.

The JpaRepository interface provides the following methods:

C	R	U	D
save(entity) saveAll(entities)	findById(id) findAll() findAllById(ids) findAll(pageable)	save(entity) saveAll(entities)	delete(entity) deleteById(id) deleteAll() deleteAll(entities)

The JpaRepository interface provides a method that returns data in the paginated form, which is as follows:

```
○ Page<T> findAll(Pageable pageable)
```

The argument is a pageable object, which can be created, as shown below.

```
○ PageRequest.of(0, 2)
○ PageRequest.of(0, 2, Sort.by("duration").ascending())
```

The first argument specifies the page number, and the second argument specifies the page size. The third argument is optional; it specifies how the sorting needs to be done, by column or by attribute. It also specifies the sorting direction.

Even though Spring Data JPA provides query methods for basic CRUD operations, they are not sufficient for real-world applications, especially the retrieve methods. For example, you can only search for a movie using its ID. You cannot use any other field, such as movie name, because JpaRepository only provides the findById() method. If you want to search for movies based on some other criteria, you will need to provide **custom query methods**.

While providing custom query methods, you need to follow certain naming conventions so Spring Data JPA can provide implementations for those methods.

Query Method	Example	Description
Movie findById(int id)	-	Find a movie based on its ID

Movie findBy Field (Type field)	Movie findByMovieName (String movieName)	Find a movie based on the movie name. This will fail if more than one movie exists with the same name.
List<Movie> findBy Field (Type field)	List<Movie> findByMovieName (String movieName)	Find all the movies based on the movie name. This is safe when the column does not have a unique constraint.
Movie findBy Field1AndField2 (Type1 field1, Type2 field2)	Movie findByMovieNameAndDuration (String movieName, int duration)	Find a movie based on the movie name and its duration. This will fail if more than one movie with the same provided criteria exists. Consider changing return type to List<Movie> for safety.
Movie findBy Field1OrField2 (Type1 field1, Type2 field2)	Movie findByMovieNameOrDuration (String movieName, int duration)	Find a movie based on the movie name or its duration. This will fail if more than one movie with the same provided criteria exists. Consider changing return type to List<Movie> for safety.

3. Data Access Layer - Mapping

Mapping between entities can be of four types:

- One to one
- One to many
- Many to one
- Many to many

These types can be further categorised into two types: unidirectional and bidirectional.

So, we can have one-to-one unidirectional mapping or one-to-one bidirectional mapping.

Representing Many-to-One relationships: entity class on the 'many' side has an attribute of the entity on the one side, and that attribute is marked with the **@ManyToOne** annotation.

```
@Entity

public class Theatre {

    @Id

    @GeneratedValue

    private int theatreId;

    ...

    @ManyToOne

    private City city;

    ...

}
```

In the above example, hibernate creates the foreign key mapping between theatre and city.

But there are two issues:

- The foreign key column is named city_city_id (mapped table name + attribute name), but we want to name it city_id.
- We want to add a constraint that city_id cannot be null.

If these were normal columns, @Column annotation could be used, but the @Column annotation will not work with foreign key columns.

To customise the foreign key column, use the **@JoinColumn** annotation.

With this annotation, the foreign key column can be customised, such as providing a custom name, adding constraints such as nullable and unique, and so on.

```
@Entity

public class Theatre {

    @Id

    @GeneratedValue

    private int theatreId;
```

```

        ...

        @ManyToOne

        @JoinColumn(name = "city_id", nullable = false)

        private City city;

        ...

    }

```

To make this relation bidirectional, provide an attribute in the entity class on the one side to refer to the entity class on the 'many' side.

That attribute would be of collection type, as the entity class on one side, that is, 'city' in this case, can be linked to many instances of the entity class on the 'many' side, that is, theatre in this case. And to store many instances, we have to use a collection.

The attribute would be marked with the **@OneToMany** annotation.

```

@Entity

public class City {

    @Id

    @GeneratedValue

    private int cityId;

    ...

    @OneToMany

    private Set<Theatre> theatres;

    ...

}

```

Hibernate creates a separate mapping table to map the city with theatre.

One city can be associated with many theatres, and you cannot store information about many entities just by adding a new foreign key column because it will violate the principles of normalisation.

But we do not want to create a separate table, as the mapping information is available in the foreign key of the theatre entity.

To tell Hibernate to use the same mapping information present in the theatre table to create a bidirectional relationship without creating a new mapping table, use the **mappedBy** property for this purpose.

With this property, we provide the name of the attribute in the entity class on the many side, which is responsible for this relationship. For example, in the theatre class, the city attribute (name of the attribute, not the type) is responsible for the one-to-many relationship with the city class.

So, in the @OneToMany annotation, provide the name of this attribute using the mappedBy property.

```
@Entity

public class City {

    @Id

    @GeneratedValue

    private int cityId;

    ...

    @OneToMany (mappedBy = "city")

    private Set<Theatre> theatres;

    ...

}
```

When Hibernate reads data from the one side table of the one-to-many relationship, it follows the lazy initialisation strategy, which means when Hibernate reads from the city table and creates objects of the city class, it does not immediately load the theatres collection of the city class objects.

So when we try to add data to these two tables, theatre and city, the following exception occurs:

org.hibernate.LazyInitializationException: failed to lazily initialize a collection of role: com.upgrad.mba.entities.City.theatres, could not initialize proxy - no Session

To resolve this, change the fetching strategy from Lazy to Eager, which means that Hibernate will load the theatre collection at the same time that it is reading the city table.

Fetching strategies are of two types: Lazy and Eager.

Lazy is the default fetching strategy for `@OneToMany` and `@ManyToMany` annotations. The Lazy strategy sends an indication to Hibernate that when it is loading entities from a table that has a one-to-many or many-to-many relationship with other tables, it should not load entities of those tables immediately but only load them into the collection when required.

On the other hand, the Eager strategy sends an indication to Hibernate that when loading entities from a table, it should load all the associated entities as well.

Eager is the default strategy for `@ManyToOne` and `@OneToOne` annotations.

Lazy is the default strategy for `@***ToMany` annotations, whereas Eager is the default strategy for `@***ToOne` annotations.

In a `@***ToMany` relationship, the associated entities would be many. Hibernate optimises this by loading the associated entities only when required. But in a `@***ToOne` relationship, the associated entity would be just one. So, Hibernate loads it immediately; so the session does not have to be kept open.

To provide a one-to-one relationship, use the **`@OneToOne`** annotation.

Take example of car and engine, they have a one-to-one relationship, and a bidirectional relationship is established as follows:

<pre> @Entity public class Car { @Id private int carId; private String carName; @OneToOne private Engine engine; } </pre>	<pre> @Entity public class Engine { @Id private int engineId; private int engineName; @OneToOne (mappedBy = "engine") private Car car; } </pre>
---	---

In a similar way, we can provide a bidirectional many-to-many relationship between the Movie and Theatre tables, using **`@ManyToMany`** annotation.

<pre> @Entity public class Movie { @Id private int movieId; private String movieName; } </pre>	<pre> @Entity public class Theatre { @Id private int theatreId; private int theatreName; } </pre>
--	---

<pre>@ManyToMany private Set<Theatre> theatres; }</pre>	<pre>@ManyToMany (mappedBy = "engine") private Set<Movie> movies; }</pre>
---	---

However, a ManyToMany relationship must have a mapping table which can be customised using the **@JoinTable** annotation in the entity that is not using the mappedBy property, that is 'movie' in this case.

If you want the mapping table to be named movie_theatre, the foreign key column for the movie table to be named movie_id, and the foreign key column for the theatre table to be named theatre_id, we can achieve that as shown below.

```
@JoinTable(name = "movie_theatre", joinColumns = @JoinColumn(name="movie_id"), inverseJoinColumns = @JoinColumn(name = "theatre_id"))
```

To establish a one-to-many relationship between a customer and their contact number without creating a separate entity table for contact numbers, as a contact number is not a separate entity, use the **@ElementCollection** annotation.

Contact numbers are part of the customer entity but cannot be stored in the same table, as we are not sure how many contact numbers will be registered by a customer.

```
@Entity
@Table(name = "Customer")
public class Customer {

    @ElementCollection (fetch = FetchType.EAGER)
    @CollectionTable(name = "customer_contact_number",
        joinColumns = @JoinColumn(name = "customer_id"))
    @Column(name = "mobile_number", nullable = false)
    private Set<Integer> phoneNumbers;
}
```

JPA Cascading

To propagate changes from an entity to its associated entities, use the **cascade** property of the relationship annotation.

Cascading can be triggered in the following six scenarios (If we perform any of the following actions on an entity, the same action will be applied to its associated entities):

- PERSIST: Saving an entity
- MERGE: Moving an entity from a detached state to a persistent state
- REMOVE: Deleting an entity
- REFRESH: Reloading an entity
- DETACH: Moving an entity from a persistent state to a detached state
- All: All of the above

```
@Entity
public class Customer {
    @OneToMany(mappedBy = "customer", fetch = FetchType.EAGER,
        cascade = {CascadeType.ALL})
    private Set<Booking> bookings;
}
```

4. Service Layer

Service Layer, or the Business Layer, is responsible for processing client requests based on the business logic and generating an appropriate response.

The **@Service** annotation is used to mark service layer classes.

This annotation does not provide any other feature and can be used interchangeably with the **@Component** annotation but using the **@Service** annotation with the service classes is considered to be a good practice.

All the classes and interfaces related to the service layer will be put inside the services package.

Provide corresponding methods in the Service interface and their implementation in a class that implements the interface.

For example: MovieService and MovieServiceImpl.

Transaction Management and @Transactional Annotation

Transactions help you treat multiple actions as a single unit. With the help of transactions, you can execute several database queries as a single query (they are not executed as a single query; this is only to understand transactions). So, either change caused by the all of the queries will be present in the database or none of them.

The main goal of a transaction is to provide **ACID characteristics** to our database, which are as follows:

- Atomicity: All or nothing (either all queries are executed or none of them gets executed)
- Consistency: A database should be consistent before and after a transaction (consistent state --transaction-> consistent state).
- Isolation: Changes made through one transaction should not be visible to other transactions until you commit that transaction.
- Durability: Once you commit the transaction, the changes will be reflected in the database.

In the Spring Boot application, you can conduct transaction management with the help of only the @Transactional annotation. This annotation can be used to mark a method or the entire class. If a class is marked with this annotation, then it is treated like all the methods of that class are marked with this annotation. All the operations inside a transactional method (a method with @Transactional annotation) are performed as a single unit of work.

```
@Transactional
@Override
public List<City> acceptMultipleCityDetails(List<City> cities)
{
    for (int i=0; i<cities.size(); i++) {
        cities.set(
            i,
            acceptCityDetails(cities.get(i))
        );
    }
    return cities;
}
```

@Transactional annotation provides several attributes which help in customising transaction management inside the application. The most important ones are as follows:

- Propagation: This property specifies whether to use an existing transaction or create a new one or decide the action to be taken when a transaction does or does not exist.

```
@Transactional(propagation = Propagation.REQUIRED)
```

Some of the propagations supported by Spring are as follows:

- REQUIRED: Either join an active transaction or start a new one. This is the default strategy for the @Transactional annotation.
- SUPPORTS: Either join an active transaction or execute without a transaction.
- MANDATORY: Either join an active transaction or throw an exception.
- readOnly: When you only want to execute the read operations (SELECT queries) inside a transaction, you can use the readOnly property and set it to true.

This will optimise the transaction management and improve the performance of the application.

```
@Transactional(readOnly = true)
```

- rollbackFor: This property helps you specify a list of exception classes for which the transaction should be rolled back.
If any of the provided exceptions occurred, then the transaction should definitely be rolled back.

```
@Transactional(rollbackFor =
{CityDetailsNotFoundException.class,
EntityNotFoundException.class})
```

- noRollbackFor: This property helps you specify a list of exception classes for which the transaction should NOT be rolled back.
Even if any of the provided exceptions occurred, then the transaction should NOT be rolled back.

```
@Transactional( noRollbackFor =
{CityDetailsNotFoundException.class,EntityNotFoundException.class})
```

Disclaimer: All content and material on the upGrad website is copyrighted material, either belonging to UpGrad or its bonafide contributors, and is purely for the dissemination of education. You are permitted to access print and download extracts from this site purely for your own education only and on the following basis:

- You can download this document from the website for self-use only.
- Any copies of this document, in part or full, saved to disc or to any other storage medium may only be used for subsequent, self-viewing purposes or to print an individual extract or copy for non-commercial personal use only.
- Any further dissemination, distribution, reproduction, copying of the content of the document herein or the uploading thereof on other websites or use of content for any other commercial/unauthorized purposes in any way which could infringe the intellectual property rights of UpGrad or its contributors, is strictly prohibited.
- No graphics, images, or photographs from any accompanying text in this document will be used separately for unauthorised purposes.
- No material in this document will be modified, adapted, or altered in any way.
- No part of this document or UpGrad content may be reproduced or stored in any other website or included in any public or private electronic retrieval system or service without UpGrad's prior written permission.
- Any rights not expressly granted in these terms are reserved.